

DOCKER

1. Qu'est-ce que Docker ?

Docker est une plateforme de **virtualisation** qui permet de créer, déployer et exécuter des applications dans un environnement isolé et portable. Chaque application est empaquetée avec ses dépendances dans un **conteneur**, ce qui garantit un **fonctionnement identique d'une machine à l'autre**.

2. Prérequis avant l'installation de Docker

Avant d'installer Docker, il est recommandé de supprimer les anciens paquets pouvant entrer en conflit avec la version officielle. Une fois cela fait, il faut mettre à jour les dépôts, installer les dépendances nécessaires, ajouter la clé GPG officielle de Docker, puis déclarer le dépôt Docker dans APT.

```
for pkg in docker.io docker-doc docker-compose podman-docker containerd
runc; do
  sudo apt-get remove -y $pkg
done

sudo apt update
sudo apt install -y ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
"deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/debian \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
```

3. Installation de Docker

Une fois le dépôt ajouté, installe Docker et ses composants principaux avec la commande suivante :

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin
```

Le service Docker peut ensuite être vérifié avec **systemctl status docker**. Si besoin, il peut être démarré manuellement avec **sudo systemctl start docker**.

Pour tester le bon fonctionnement de l'installation, exécute :

```
sudo docker run hello-world
```

Si le message de confirmation s'affiche, Docker est correctement installé.

4. Commandes importantes de Docker

Commande	Explication
docker pull [image]	Télécharge une image depuis Docker Hub.
docker images	Liste les images présentes localement.
docker run -d -p [port_hôte]: [port_conteneur] --name [nom] [image]	Lance un conteneur à partir d'une image.
docker ps	Affiche les conteneurs actifs.
docker ps -a	Affiche tous les conteneurs, y compris arrêtés.
docker exec -it [conteneur] bash	Ouvre un terminal dans un conteneur en cours d'exécution.
docker stop [conteneur]	Arrête un conteneur.
docker start [conteneur]	Redémarre un conteneur.
docker rm [conteneur]	Supprime un conteneur.
docker rmi [image]	Supprime une image locale.

5. Création d'images personnalisées avec un Dockerfile

Docker permet de créer des images sur mesure à l'aide d'un fichier nommé **Dockerfile**.

Ce fichier contient les instructions nécessaires pour construire l'environnement de l'application.

Exemple : Image "Hello World"

Crée un fichier **Dockerfile** contenant :

```
FROM debian:latest
CMD ["echo", "Hello World!"]
```

Puis construis l'image avec :

```
docker build -t mon-image-hello
```

Enfin, exécute-la avec :

```
docker run mon-image-hello
```

6. Automatisation avec Docker Compose

Docker Compose sert à définir et lancer plusieurs services dans un seul fichier YAML.

Il simplifie le déploiement d'une application multi-conteneurs grâce à une configuration centralisée des services, réseaux et volumes.

Pour l'utiliser :

1. Créer un répertoire de travail.
2. Créer un fichier **docker-compose.yml**
3. Définir les services, ports, volumes et variables d'environnement.
4. Lancer l'ensemble avec **docker compose up**

Exemple simple :

```
services:
  web:
    build: .
    image: mon-image
    ports:
      - "8080:80"
```

Commandes utiles :

- **docker compose build** : construit les images.
- **docker compose up** : lance les services au premier plan.
- **docker compose up -d** : lance les services en arrière-plan.
- **docker compose stop** : arrête les services.